

Designing an FPGA Inference Kernel for FlashAttention

Ismael Arechiga Duran

iauredur@stanford.edu

github.com/DonIsmaelito/flashattention_kernel

1. Introduction

I wanted to understand the hardware stack better. Most of my experience is a few levels up, and I kept wondering what happens at the hardware level, where a computation becomes cycles, memory accesses, and wires. I also wanted to learn the AI side more deeply. Self-attention sits right at the intersection. It is the core of every transformer, and it is also a concrete thing that someone has to build in hardware, so I picked it and tried to design an accelerator for it from scratch.

I had not written FPGA hardware before, so most of the work was reading: the attention papers [1, 2], the board’s documentation [3], and the high-level synthesis toolchain [4, 5]. This report follows that path. It goes through what I read, what each source told me, and the design choice that followed. It is a proposed design, not a finished chip: everything here is modeled and simulated, not yet synthesized.

Self-attention is the core operation in transformer models [2]. Given matrices $Q, K, V \in \mathbb{R}^{N \times d}$, it computes

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \quad (1)$$

which needs an $N \times N$ score matrix $S = QK^T$ as an intermediate, and whose cost grows quickly with the sequence length. For my application $N = 512$ is the sequence length and $d = 64$ the head dimension, with a single head and batch 1. The raw compute is modest; what decides whether the design runs fast or stalls is the data movement pattern. My goal was latency: run attention over the 512-token window in under 200 us with a deterministic execution time. That target comes from the anomaly detection use case (Section 9), where detection has to finish within the trigger interval or the monitor goes blind between windows.

1.1. The problem I aimed at

To make any of this concrete I first needed a real workload with a real latency target, and network intrusion detection is a good one. Most intrusion detection

today is rule-based: a packet that matches a known-bad signature gets flagged. That catches attacks that have been seen before but misses anything new, including an attack spread across many small steps that each look normal on their own.

I wanted to see whether attention could catch those cases. Instead of checking packets against rules one at a time, the idea is to look at a window of recent packets together and ask whether the sequence as a whole looks normal. Self-attention fits this because it compares every packet in the window against every other packet, which is what lets it pick up multi-step patterns.

This is also what sets the problem size. A 10 Ggabit per second (Gbps) link produces roughly 2.5M packets/sec, so a detector keeps a sliding window of recent packet embeddings and runs attention over it. GPU-based approaches batch 10K to 100K packets and report a result with 50 to 500 ms of delay, which is fine for after-the-fact analysis but too slow to act on traffic as it arrives. I settled on a 512-packet window and a 200 us target, the kind of latency real-time use would need.

1.2. Why an FPGA

With that problem in hand, the next question was what to run it on, and I had not used an FPGA before, so I had to work out whether one even fit. An FPGA is a chip whose logic you lay out yourself: instead of running software on a fixed processor, you describe the circuit you want and let the tools build it. Reading about them, two things lined up with my problem.

The first is that you get to choose how each number is stored, rather than being handed 32-bit floating point whether you need it or not. My data does not really call for floats anyway. The packet fields it starts from, things like port numbers and byte counts, are whole numbers in a known range, and the vectors built from them stay bounded, so I could keep small values small and spend more bits only where the math actually needs them. I work out the exact sizes later; the

freedom to choose them is part of why an FPGA fit.

The second is that the whole computation is small enough to keep on the chip itself. Everything one attention pass touches comes to about 256 KB, a small fraction of the chip’s fast internal memory, so once it is loaded nothing has to travel back out to the slower memory on the board while the work runs. That matters because attention is limited mostly by data movement, as the next section explains.

Those two properties are why I chose an FPGA. Because I am laying out the circuit myself, every run takes the same amount of time, and I can build the whole thing as one continuous pipeline rather than a sequence of steps that keep handing data back to memory.

2. Background and Related Work

2.1. The memory cost of attention

Standard attention stores the full $N \times N$ score matrix $S = QK^T$ in memory so it can normalize with softmax. That storage grows with the square of the sequence length and, at small batch sizes, makes the operation memory-bound. For $N = 512$ with 16-bit elements, S is 512 KB. The bigger cost is the traffic pattern. The hardware writes the score matrix S , reads it back to apply softmax, writes the resulting probability matrix P , then reads P back to multiply by V , which is four passes over the full matrix.

2.2. FlashAttention

The FlashAttention paper [1] pointed the way. It tiles Q and K into blocks that fit in GPU SRAM and uses an online softmax, so the full score matrix never has to exist, which sharply cuts the slow memory accesses. Attention is limited by data movement, not by arithmetic, so the main job is to keep the data close to the compute.

2.3. FPGA Attention Accelerators

I also looked at how other people had put attention on FPGAs. Most of that work reduces the compute through approximation, by pruning tokens, quantizing, or moving into the frequency domain. I kept exact attention instead and reorganized the memory access pattern to keep everything in BRAM, which is essentially what FlashAttention does with GPU shared memory. I did this partly so the output would match a standard software reference exactly, which made it easier to tell whether the hardware was correct while I was still learning the tools.

3. Workload Characterization

3.1. Compute Requirements

Before designing the hardware I added up how much arithmetic the kernel actually does, since that sets how many multiply units it needs. With 32 by 32 tiles, the 512-token sequence splits into 16 tiles each way, giving 256 tile-pair iterations. Each pair multiplies a Q tile by a K tile and then weights a V tile, and altogether the kernel does about 33.6 million multiply-accumulates, or roughly 68.5 million FLOPs once softmax is included.

3.2. Data Movement Analysis

There are two kinds of data movement to account for. The first is off-chip, between the host and the FPGA: Q is 64 KB, and K , V , and O are each the same, so 256 KB in total, small enough to fit entirely on-chip. The second is on-chip, from BRAM to the compute units: each tile-pair iteration moves a 4 KB Q tile, a 4 KB K tile, a 4 KB V tile, and an 8 KB O tile read and written, about 20 KB in all. Across all 256 iterations that is roughly 5 MB of on-chip traffic.

Each Q row is reused across all 16 K/V tiles, and each K and V vector is used by all 32 Q rows in the tile. So the design does a lot of math for every byte it reads, an arithmetic intensity of about 131 operations per byte, far more than the chip’s memory has to keep up with.

3.3. The bottleneck

In standard attention the limit is off-chip traffic for the score matrix. Once I tile everything to fit on-chip, that traffic disappears and HBM bandwidth stops mattering during compute, so the bottleneck moves inward to how fast I can read BRAM. The 16 PEs each want 64 values every cycle, which works out to 2,048 bytes per cycle. A single BRAM cannot supply that, so the data has to come from many banks at once, which is why partitioning (Section 5) is the key optimization.

3.4. Host CPU/FPGA Partitioning

The FPGA handles the self-attention computation and the on-chip window buffer storage. Those benefit from custom precision, pipelining, and keeping the data on-chip. The host CPU handles packet parsing, feature extraction, the embedding and QKV projection, the classification MLP, the trigger logic, and alerting. Those are standard matrix-vector multiplies and comparisons, where a CPU is equally fast and much simpler to implement.

4. SystemC Microarchitecture

4.1. Design Methodology

I wrote the design in synthesizable SystemC, which is C++ with hardware semantics that an HLS tool can turn into a circuit. For the wiring between modules I used MatchLib [4], and for the fixed-point math I used Algorithmic C (ac_datatypes). Every module is built the same way: a constructor with an SC_THREAD, reset code for its ports and state, and a main loop that advances a cycle at a time with wait(). Because the modules only talk through MatchLib channels, a stall in one flows back to the others on its own, and I can test each module by itself.

4.2. Module Hierarchy

The piece I actually send to synthesis is FlashAttnKernel, and Fig. 1 shows what is inside it.

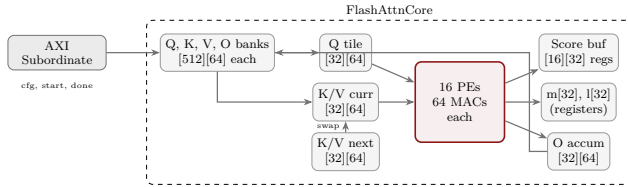


Figure 1. FlashAttnCore microarchitecture. 16 PEs read Q and K tiles from partitioned BRAM, compute scores, update online softmax state (m, l in registers), and accumulate weighted V into the output buffer. K/V tiles are double-buffered (curr/next).

The rest of the system sits around it but is not synthesized. A NetworkAnomalyDetector wraps the FlashAttnCore attention engine together with the parts that feed it: a TriggerController that decides when to run, an EmbeddingProjection that turns packets into vectors, three CircularWindowBuffers holding the recent Q, K, and V, and a ClassificationHead that scores the result.

4.3. Processing Element Architecture

The core of the design is a row of 16 processing elements, or PEs, working in parallel. Each PE computes one 64-element dot product per cycle, fully unrolled so all 64 multiplies happen at once. The K-column loop feeds them 32 columns at one column per cycle (#pragma hls_pipeline_init_interval 1). Since a tile has 32 query rows and I have 16 PEs, each tile-pair takes two passes, and each pass runs about 32 cycles for the scores plus 4 for softmax, so roughly 72 cycles per tile-pair.

Adding 64 products into one number is itself a small tree, six levels deep, each level summing pairs of 24-bit

values. Done all at once that tree is too slow for 250 MHz, so I pipelined it into three stages of two levels each. That adds two cycles of latency but lets a new dot product start every cycle.

4.4. Tiled Attention Loop

The main loop mirrors FlashAttention Algorithm 1 [1]. For each Q-tile it loads the tile, resets the softmax state ($m = -\infty$, $\ell = 0$, $O = 0$), and preloads the first K/V tile. It then walks the K/V tiles, and for each one it loads the next tile while the 16 PEs process the current one over 32 K-columns, updating the online softmax and accumulating weighted V, before swapping the buffers. After the last tile it normalizes each row and writes it back.

4.5. Online Softmax Implementation

Online softmax lets me skip the full $N \times N$ score matrix. For each query row, as each new K/V tile arrives, the kernel scores the tile, finds the largest score so far, and uses it to rescale both the running output and the running sum before folding in the new exponentials. After the last tile it divides each row by its running sum. The result is exactly what plain softmax would give, computed a tile at a time.

4.6. Piecewise-Linear Exponential Approximation

Softmax needs an exponential, which is expensive to build in hardware, so I approximated it. ExpApprox replaces e^x with four straight-line segments over $[-8, 0]$, clamped to 1 above 0 and to 0 below -8 . Each segment is just one multiply, one add, and a comparison to pick it. That is good enough here, because softmax only cares about relative sizes, and after the maximum is subtracted any input below -8 contributes essentially nothing.

5. HLS-Aware Design and Optimization

5.1. BRAM Array Partitioning

A single BRAM block has two ports, so reading the 64 elements of a Q row would take 32 cycles. I instead apply cyclic partitioning with factor 64 on the head dimension to each tile array (q_tile, the current and next K and V tiles, and row_out). This puts every head-dimension element in its own bank, so an entire row is readable in one cycle. The score buffer and the softmax state (row_max and row_sum) are small enough to live entirely in registers.

5.2. Pipeline Pragmas and Initiation Interval

The design uses three levels of parallelism. A full unroll (#pragma hls_unroll yes) makes the 64-element

Table 1. Cycle Breakdown per Q-Tile ($N = 512$, 250 MHz)

Phase	Cycles	Time (μ s)
Q tile load (B_r rows)	32	0.13
Reset softmax state	1	0.004
K/V preload (first tile)	32	0.13
Inner loop (16 tiles, dbl-buf)	1,152	4.61
Finalize (normalize + writeback)	69	0.28
Per Q-tile total	1,286	5.14
Full kernel (16 Q-tiles)	20,576	82.3

dot product, the scale-accumulate, the row-max, and the exp operations fully spatial, which gives 64 parallel multipliers per PE. A pipeline at $II=1$ (`#pragma hls_pipeline_init_interval 1`) feeds one new column per cycle through the K-column loop, and the tile loads and finalization target $II=1$ as well. Sequential batching runs the Q-row loop as two batches, and within each batch 16 PEs work on 16 rows in parallel over the same K columns. The target II is 1 for the K-column and dot-product loops.

Table 1 shows the per-phase cycle breakdown from the analytical latency model (`estimate_latency.py`).

At 20,576 cycles and 250 MHz, the kernel finishes in 82.3 μ s, an effective throughput of about 0.83 TFLOPS.

5.3. Double-Buffering of K/V Tiles

To hide the load time I keep two copies of each K/V tile. While the PEs work on the current tile, I load the next one into a second pair of buffers in the background. When the PEs finish, a short pipelined loop copies the next buffers into the current ones, which acts as the swap.

Without double-buffering, each iteration would load and then compute, costing about 104 cycles. With it, the load overlaps with compute, so each iteration after the first costs only its 72 compute cycles. Across the kernel this saves about 7,680 cycles, cutting total runtime by 27% (28,256 cycles down to 20,576).

5.4. Parallelism, resources, and clock

PE count trades off directly against resources and latency. A single PE would be far too slow, while 256 PEs would need 16,384 DSPs, far past the VU47P budget. I picked 16 PEs as the balance. Their 1,024 DSPs are 11.3% of the VU47P, leaving room for the softmax logic and routing, and the resulting 82.3 μ s meets the 200 μ s target with headroom for a second head. Going wider would also strain timing, since the reduction tree

already pushes 250 MHz and more PEs would make routing harder.

5.5. Synthesis Status

I was not able to run HLS synthesis: the Catapult Ultra license server was unavailable while I was working on this, so every resource and timing number in this report is analytical rather than measured. Running synthesis is the first thing I would do next.

6. Memory System and Data Movement

6.1. On-chip residency

The whole working set is small. Q, K, V, and O are 64 KB each, so 256 KB in total, which is under 3% of the chip’s BRAM. The only time anything crosses to off-chip memory is the initial load and the final write-back. During the 20,576-cycle compute phase, nothing goes to or from HBM at all.

6.2. On-Chip Bandwidth Analysis

During the score computation each PE reads a 64-element Q row every cycle, about 2,048 bytes across the 16, plus a single shared 128-byte K column, while the score writes stay in registers. That is roughly 2,176 bytes per cycle out of BRAM, and the Q tile dominates because it needs 16 separate row reads at once.

6.3. AXI Interface

The kernel talks to the host over AXI, the standard bus protocol Xilinx parts use (`AxiSpec.h`). The link is 128 bits wide and can move up to 256 words in a single burst. For now I only wired up register access: the host writes the config and a start signal and reads back the status, checksum, and cycle count, while the data itself sits in fixed on-chip arrays for simulation. In a real deployment I would add a bus master to burst the inputs straight from HBM into the tile buffers.

6.4. F2 Platform Bandwidth Limits

The F2’s PCIe Gen4 x16 link gives about 25 GB/s, so the 256 KB payload takes 10.5 μ s, a small part of the end-to-end latency. Its HBM2 gives 460 GB/s across 32 channels, far more than needed since all data lives on-chip during compute. What matters is on-chip BRAM: with 64-way partitioning each array delivers 128 bytes per cycle, or 32 GB/s at 250 MHz, and the six partitioned arrays together supply about 192 GB/s, which matches the compute demand. PCIe and HBM are each used once, for the bulk load and the readback, so external bandwidth never limits throughput.

6.5. Circular Window Buffer

The CircularWindowBuffer stores packet embeddings in a transposed layout, `data_bank[64][512]` instead of the usual `[512][64]`, so all 64 dimensions can be read in one cycle, which is what the dot-product units need.

Circular addressing is free since $N=512$ is a power of two. On a trigger the buffer freezes by saving the current write pointer as `frozen_base`, which reads then use as the logical origin so attention sees a consistent snapshot. A write-stall pulls `write_enable` low while the engine is computing and classifying, so new packets cannot overwrite what it is reading. Without the stall, at 2.5M packets/sec, about 206 packets would arrive during the 82.3 us window and corrupt the buffer, so a 256-entry input FIFO (16 KB) absorbs them in the meantime.

6.6. Standard vs. Fused Off-Chip Traffic

Standard attention moves about 2,304 KB off-chip for the 512-token window, because it writes and reads the full score and probability matrices. The fused approach moves only the 256 KB of Q, K, V, and O, which is 9 times less. That 256 KB fits on-chip regardless, but the standard write-read-write-read pattern would create BRAM port contention, and at longer sequences the score matrix grows too large for BRAM and forces off-chip spills that the tiled approach avoids entirely.

7. Programming Model

7.1. Host-Side API

The host-side interface is small; it lives in `flash_attn_api.h`. To run one attention pass the host opens a handle, loads Q, K, and V, sets the config, starts the kernel and waits for it to finish, then reads back the output and a status word and closes the handle. That is the whole API.

7.2. Mapping attention to the kernel

One accelerator invocation computes one full self-attention pass, from Q, K, and V to the output O. This is one attention head of one transformer layer. For a multi-head, multi-layer model, the host invokes the kernel once per head per layer, running the feed-forward parts on the host between layers.

7.3. Effects on the host software

The fixed tile sizes are a help here. B_r , B_c , d , and N are all set at compile time, so the host just sets `seq_len` and the kernel handles partial tiles internally, leaving almost nothing to configure.

Table 2. Estimated FPGA Resource Utilization (VU47P)

Resource	Used (est.)	Available	Util.
DSP48E2	1,688	9,024	18.7%
BRAM36K	83	2,016	4.1%
URAM288K	6	1,280	0.5%
LUT	~17,000	432,000	~4%
FF	~108,000	864,000	~13%

The fixed-point types, on the other hand, make data preparation harder. The host has to convert `FP32` to `ac_fixed<16,6,true>` before sending it over, and getting the quantization wrong produces silently wrong results. I thought about adding an on-chip converter but decided it was not worth the complexity for a single-head design.

The single head is the main limit on throughput. Multi-head needs one sequential invocation per head, each paying the PCIe cost, so eight heads cost about 88 us of transfer overhead, which hardware replication would fix.

Keeping everything on-chip also keeps the streaming model simple. There is no tiled DMA engine to build, so the host does one bulk load, triggers compute, and reads the result, which is much simpler than designs that overlap data movement with compute.

8. FPGA Resource Utilization and Timing

8.1. Resource Estimates

Table 2 shows analytical resource estimates from `estimate_resources.py`, counting hardware primitives bottom-up.

The DSPs break down as 1,024 for the dot products (16 PEs at 64 MACs each), 16 for the exp units, 512 for scale-accumulate (time-shared with Stage 1), 8 for miscellaneous logic, 64 for the embedding, and 64 for the MLP.

The BRAMs break down as 60 for the four sequence banks (64 KB each), 7 for the tile and double buffers, 7 for the embedding weights, 4 for the MLP weights, 4 for the input FIFO, and 1 for the reciprocal LUT. The three circular window buffers (192 KB) go to URAM.

LUT and FF counts are rough estimates based on control logic complexity and pipeline register depth. Accurate numbers require synthesis. The AWS F2 shell uses about 12% of FPGA resources, leaving 88% for custom logic, and these estimates fit comfortably in the remaining budget.

8.2. Clock Frequency Analysis

My clock target is 250 MHz, which leaves 4 ns per cycle. The tightest path is that six-level addition tree after the multiplies; combined it would run about 6 ns, too slow, which is why I split it into pipelined stages of roughly 2 ns each. I cannot confirm 250 MHz without synthesis, but even if it only reached 200 MHz the latency would be about 103 us, still under the 200 us target.

8.3. Scaling Limits

Three things limit the design, from most to least pressing. BRAM port bandwidth comes first, since at 16 PEs the partitioned BRAMs are near saturation and 32 PEs would need proportionally more banks and routing. Routing congestion comes next, because above 50% DSP utilization Vivado has trouble routing the short paths between DSPs and BRAMs, which hurts the clock frequency. The DSP count is the hard limit, since 256 PEs would need 16,384 DSPs, 1.8x more than the VU47P has.

9. Application: Network Anomaly Detection

9.1. System Architecture

I split detection into two tiers so the cheap check runs on every packet and the expensive one runs only when it has to. The first tier looks at each packet on its own. It is a combinational filter (Tier1Screening) that checks five rules at once: low TTL, a suspicious port range, a SYN-scan pattern, a high packet rate, and an oversized payload. It takes one cycle and about 500 LUTs with no DSPs, and if any rule fires the packet is flagged with a reason code. The second tier is the sliding-window attention. It runs every 500 packets, or right away when the first tier flags something, over the latest 512 embeddings. This is the heavy compute the FPGA handles.

9.2. The trigger controller

I wrote the TriggerController as a six-state machine. It waits idle until the host starts it, then arms and counts packets while screening each one, firing either on a schedule (every 500 packets) or the moment the first tier flags something. On a trigger it drains the embedding pipeline and freezes the circular buffers so attention sees a consistent snapshot, runs attention with writes disabled, scores the result with the classifier, and compares that score to a threshold, raising an interrupt if the window looks anomalous before going back to the armed state.

9.3. Embedding and Classification Pipeline

EmbeddingProjection turns a 32-element feature vector into the Q, K, and V vectors (64 each) with four matrix-vector multiplies. I reuse a single 64-DSP dot unit across all four, pipelined so it takes about 260 cycles per packet, with 28 KB of weights.

ClassificationHead is a small two-layer MLP that turns the attention output into a single anomaly score. It averages all N output rows into one 64-element vector, then runs that through a 64-to-128 layer with a ReLU, and finally a 128-to-1 layer for the scalar score. A 512-entry reciprocal lookup table does the averaging division, so I do not need a hardware divider.

In a real deployment these would run on the host, but keeping them in SystemC let me validate the full pipeline end-to-end in simulation before dealing with the host/FPGA boundary.

9.4. End-to-End Latency Budget

Adding the pieces up, the end-to-end budget is about 99 us. Host packet parsing and embedding take 2 to 5 us, writing QKV over PCIe takes 10.5 us, the attention compute takes 82.3 us, reading the output back is about 3 us, and the classifier adds roughly 1 us. At that budget I can fit two sequential heads (198 us) and still land under the 200 us target.

10. Design Tradeoffs

I chose online softmax over a two-pass scheme. The alternative would buffer each full score row (1 KB) and do softmax in two passes, which saves about 30% of the softmax DSPs by removing the rescaling logic. I went with online softmax because it generalizes to any sequence length without a row-buffering constraint and matches FlashAttention directly.

For precision, I went with a fixed-point ladder instead of uniform FP16. The 16/24/24/32-bit ladder uses fewer DSPs per operation than FP16, at the cost of doing the range analysis by hand. The 6 integer bits give a range of $[-32, 32]$, sized for normalized feature embeddings, and overflow saturates through AC_SAT rather than wrapping.

The tile size landed at 32x32. Tiles of 64x64 would cut the loop overhead by 4x but need 4x more BRAM and PEs. At 32x32 the loop overhead stays under 5% of total cycles, the resources stay moderate, and 32 divides 512 evenly.

I validated a single head before worrying about replication. Multi-head just replicates FlashAttnCore, reaching about 45% DSP at four heads, and the tiling and softmax correctness do not depend on head count.

11. Conclusion

FlashAttention showed that the bottleneck in attention is data movement, not compute. Tiling to fit in fast memory and using online softmax to avoid storing the intermediates gives the same result with far less traffic.

These ideas port directly to an FPGA. My SystemC design tiles attention into BRAM with 32x32 tiles partitioned for single-cycle access, uses online softmax to skip the N-by-N score matrix, and fuses scoring through accumulation in one pipeline with double-buffered loads. Everything stays on-chip, execution is deterministic at 20,576 cycles, and precision is tuned per stage.

For network anomaly detection, the design targets sub-200 us detection latency with deterministic timing: because I lay out the circuit, every run takes the same number of cycles. On the analytical estimates it also fits comfortably within the chip’s resource budget.

11.1. Future work

Because the design is modeled but not yet synthesized, the clearest next step is to run Catapult synthesis. The design is already written for it, and the open question is whether the 250 MHz target holds or whether I would need to back off to 200 MHz. After that, the path to real hardware runs through the full flow from HLS to RTL to bitstream, plus training the model on labeled traffic such as CICIDS2017 and writing the host driver to run it.

References

- [1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in Proc. NeurIPS, 2022.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in Proc. NeurIPS, 2017.
- [3] Amazon Web Services, “Amazon EC2 F2 instances,” <https://aws.amazon.com/ec2/instance-types/f2/>, 2024.
- [4] NVlabs, “MatchLib: SystemC/C++ library for high-level synthesis,” <https://github.com/NVlabs/matchlib>, 2023.
- [5] Siemens EDA, “Catapult high-level synthesis,” <https://eda.sw.siemens.com/en-US/ic/catapult-high-level-synthesis/>, 2024.